HelixQA Go orchestrator — wiring plan vs Helix OTA (audit AB-G5)

Revision: 1 **Last modified:** 2026-06-23T14:50:00Z **Scope:** wire `submodules/helixqa/cmd/helixqa` (Go orchestration engine) against the live Helix OTA control plane. Investigation-only; no commit (conductor commits).

1. Current state (captured fact — see `build_vet_test.txt`)

- `submodules/helixqa` is module `digital.vasic.helixqa` (go 1.26). Engine packages: `pkg/orchestrator`, `pkg/testbank` (incl. `dispatch.go`), `pkg/detector`, `pkg/validator`, `pkg/evidence`, `pkg/reporter`, `pkg/conduit` (the §11.4.116 sync channel), `pkg/infra` (containers boot), `pkg/autonomous` (LLM-driven), ~25 vision binaries in `cmd/`.
- **`go build ./...` FAILS** (65 error lines, go.work active) AND **`GOWORK=off go build ./...` FAILS** (23 error lines). **The `helixqa` binary does NOT build.**
- `go vet ./...` and `go test ./...` are **also blocked** — every `_test.go` transitively imports the missing deps.

Root cause (FACT, not guess)

`helixqa/go.mod` declares 8 own-org `replace` directives pointing at sibling paths `../<name>`. Resolved from `submodules/helixqa/`, that is `submodules/<name>`. Presence:

replace dep	expected path	present?
`digital.vasic.containers`	`submodules/containers`	**MISSING**
`digital.vasic.challenges`	`submodules/challenges`	present
`digital.vasic.docprocessor`	`submodules/doc_processor`	**MISSING**
`digital.vasic.llmorchestrator`	`submodules/llm_orchestrator`	**MISSING**
`digital.vasic.llmprovider`	`submodules/llm_provider`	**MISSING**
`digital.vasic.security`	`submodules/security`	**MISSING**
`digital.vasic.visionengine`	`submodules/vision_engine`	**MISSING**
`digital.vasic.llmsverifier`	`submodules/llms_verifier/llm-verifier`	**MISSING**

7 of 8 own-org Go-module dependencies are absent from `submodules/`. The one present sibling (`challenges`) itself `replaces ../containers`, so it is also unbuildable standalone. **`containers/go.mod` DOES exist — but at the REPO ROOT (`helix_ota/containers`), not at `submodules/containers`** where helixqa's replace expects it. This is a §11.4.28(C) dependency-layout mismatch (the OTA project laid `containers` at root while helixqa expects it as a flat sibling under `submodules/`).

This is exactly audit AB-G5: the Go orchestrator is powerful but uninstalled vs OTA — it cannot build in this checkout.

2. How `cmd/helixqa` is invoked against a bank + live target

The CLI (`cmd/helixqa/main.go`, v0.2.0) has subcommands: `run`, `list`, `report`, `autonomous`, `http`, `replay`, `signoff`, `version`.

Two relevant entry points for OTA:

(a) `helixqa http` — for `http:`-action banks (NOT the OTA bank's shape)

`cmd/helixqa/http.go` runs a bank's `ActionTypeHTTP` steps against a live server with no browser/LLM:

```
helixqa http --bank <bank.yaml> --base-url http://127.0.0.1:8080 \
  [--admin-user U --admin-pass P] [--login-path /api/v1/auth/
login] \
  [--token-field session_token] [--timeout 30s] [--json]
```

Drives `autonomous.HTTPExecutor`; per-case PASS only when every `http:` step passes; exit non-zero on any FAIL. **Base URL is a required flag, never hardcoded** (§11.4.28 decoupling).

(b) `helixqa run` / dispatcher — for `dispatches_to:` banks (the OTA bank's shape)

The orchestrator (`pkg/orchestrator`) + `pkg/testbank.Dispatcher.Run()` execute each case's `dispatches_to:` script via an injected `DeviceExec` hook, capture the real exit code (0=PASS), enforce the `required_evidence` ledger (§11.4.69), and emit verdicts on a `conduit` JSONL sync channel (§11.4.116). This is the native equivalent of the shell `run_bank.sh`.

3. OTA bank format — MATCH vs adapter

`tools/helixqa/banks/helix_ota.yaml`: **5 cases, all `dispatches_to:` style, ZERO `http:` action steps.** The schema (`pkg/testbank/schema.go`) DOES support `dispatches_to`, `challenge_id`, `required_evidence`, `domains`, `metadata` — **the OTA bank's keys parse natively, no schema adapter needed.**

The gap is the EXECUTION HOOK, not the format: - `helixqa http` does NOT apply (no `http:` steps in the OTA bank). - `helixqa run` applies, BUT the `cmd/helixqa` CLI does **not** wire a `DeviceExec` for `dispatches_to` (grep found no `NewDispatcher`/`DeviceExec` wiring in `cmd/`). The `Dispatcher` is a library type that a consumer wraps; the OTA project would need a thin host-exec adapter (run the `dispatches_to` path via `os/exec` from the project root) injected as `DeviceExec`, plus a `Conduit` for the evidence/sync channel.

Net: bank FORMAT matches; a small CLI/host-exec **adapter** is required to run a `dispatches_to` bank from `cmd/helixqa` (or a tiny project-side Go `cmd/` that imports `pkg/testbank` + `pkg/orchestrator` and injects a host `DeviceExec`).

4. What the Go orchestrator adds over shell `run_bank.sh`

`tools/helixqa/run_bank.sh` already dispatches the 5 OTA challenges. Native `cmd/helixqa` adds: - **Native crash/ANR/issue detection** (`pkg/detector`, `pkg/issuedetector`) on captured output. - **Per-step structured validation** (`pkg/validator`) beyond shell exit code. - **Evidence ledger gate** (`pkg/testbank` `required_evidence` → real-artifact resolution, §11.4.69) enforced in-process. - **Real-time conductor⇄framework sync channel** (`pkg/conduit` JSONL events + atomic status snapshot, §11.4.116) — verdict events carry evidence paths. - **Structured `QAReport`** (`pkg/reporter`) with HTML/PDF, vs shell text. - **Autonomous LLM-driven QA sessions** (`pkg/autonomous`) and ~25 vision oracles (`cmd/helixqa-*`) for §11.4.107/.117 liveness/OCR.

5. Concrete wiring steps (to make it runnable vs OTA)

1. **Fix dependency layout** so helixqa builds. Either: (A) add the 7 missing own-org modules as siblings under `submodules/` per §11.4.28(C) + §11.4.31 `helix-deps.yaml` (`git submodule add` of `containers`, `doc_processor`, `llm_orchestrator`, `llm_provider`, `security`, `vision_engine`, `llms_verifier`); OR (B) re-point helixqa's `replace` / add a `go.work use` entry so `digital.vasic.containers` resolves to the existing repo-root `helix_ota/containers` (and likewise lay the others). Option A is the constitution-canonical layout; option B is a faster local bridge. **This is the blocking step — until it is done, no `helixqa` invocation runs.**

2. Build the binary: `cd submodules/helixqa && go build -o bin/helixqa ./cmd/helixqa`.

3. Boot the live OTA server (the OTA bank header already documents this):

```
cd server && HELIX_ADMIN_USERNAME=admin@helix.test
HELIX_ADMIN_PASSWORD=s3cret \
  HELIX_TOKEN_SECRET=test go run ./cmd/ota-server &
until curl -fsS http://127.0.0.1:8080/healthz; do sleep 0.5;
done
```

4. Add a `DeviceExec` host-exec adapter (run `dispatches_to` via `sh -c` from `helix_ota/` root, env `HELIX_BASE_URL=http://127.0.0.1:8080`) and invoke the dispatcher over `tools/helixqa/banks/helix_ota.yaml`. (A sibling stream owns the live thinker/OTA boot; defer the live run to them.)

5. Evidence lands under `docs/qa/<run-id>/` per §11.4.83; conduit JSONL under the configured evidence dir.

6. Honest gaps (§11.4.6)

- **Not yet runnable vs OTA**: the orchestrator does NOT build in this checkout (7 missing sibling modules) — captured fact, not estimate. No self-test / dry-run could be run because the binary does not compile; running `helixqa version` / any subcommand is impossible until step 5.1 is done.
- The `cmd/helixqa` CLI has **no `dispatches_to` run path wired** today — the dispatcher is a library; an adapter (≈1 small Go file injecting a host `DeviceExec`) is the missing glue even after the build is fixed.
- `containers` exists at repo root but at the WRONG path for helixqa's replace — a layout decision (§11.4.28(C)) the conductor/operator must resolve (option A vs B above) per §11.4.66.
- Live-OTA run deferred to the sibling that owns the thinker/boot — NOT exercised here.